Temas dos Seminários

Nos seminários cada grupo deverá primeiramente introduzir, e explicar os conceitos do seu tema, e, posteriomente o relacionar com alguma das partes dos trabalhos práticos desenvolvidos, quando aplicável.

Grupo 1: *Introdução, Arquitetura e Tipos de Linguagens*

Diferença entre execução direta e geração de código.

Onde o trabalho prático da disciplina se encaixa na arquitetura.

Exemplos, vantagens e desvantagens de linguagens compiladas, interpretadas e híbridas.

Grupo 2: A Cadeia de Geração Final (Fonte ao Executável)

Visão da cadeia completa até o executável.

Ferramentas fundamentais: *assemblers* e *linkers*.

Como múltiplos arquivos e bibliotecas são unidos no produto final.

Grupo 3: Estratégias de Parsing (Gramáticas LL vs. LR)

Diferença entre abordagens *top-down* (LL) e *bottom-up* (LR).

Parsing recursivo descendente e os fatores que tornam uma gramática não LL(1).

Vantagens e limitações de cada abordagem de análise sintática.

Grupo 4: Análise Semântica e Tabela de Símbolos

O que é e como estruturar uma tabela de símbolos.

Como a análise semântica verifica escopo, declaração duplicada, checagem de tipos e uso de variáveis.

Exemplos práticos de erros semânticos comuns.

Grupo 5: Representação Intermediária e Otimização de Código

O que é o código de três endereços, por que usá-lo e exemplos típicos.

Em qual etapa ocorre a otimização e o que significa otimizar (constantes, *dead code*, *strength reduction*).

O papel do código intermediário como ponte e como a dependência de variáveis se relaciona com a alocação de registradores via **coloração de grafos**.

Grupo 6: Ambiente em Tempo de Execução (Runtime)

Como variáveis e estruturas de controle são gerenciadas na memória durante a execução.

Gerenciamento de Pilha (*Stack*), *Heap* e o registro de ativação nas chamadas de função.

Grupo 7: Tratamento e Recuperação de Erros em Compiladores

A diferença crucial entre apenas detectar um erro e conseguir se recuperar dele para continuar o *parsing*.

Como os compiladores lidam com erros nas diferentes fases (léxica, sintática, semântica).

Estratégias avançadas (modo pânico, recuperação a nível de frase) para tornar o compilador robusto.